

Agent Harness Benchmark — Mimir vs Claude Code

Run set `full-suite-20260912`

Tasks: **6**

Repeats per task: **3**

Total runs compared: **36** (18 per agent)

Mimir model: **claude-sonnet-5** (mimir 0.5.0)

Claude Code model: **sonnet** (2.1.269)

Benchmark commit: **56d9ba28**

Executive summary

Metric	Claude Code	Mimir	Δ Mimir vs Claude Code
Solve rate	100.0%	100.0%	0.0%
Mean combined score	1.000	1.000	0.0%
Median elapsed time	40.15s	30.44s	-24.2%
P95 elapsed time	64.49s	70.91s	+10.0%
Mean total tokens	447,913	69,434	-84.5%
Elapsed stdev	10.33s	17.03s	+64.8%
Scope violations	0	0	n/a
Adapter failures	0	0	n/a

Headline metrics Source: summary.json

Figures computed directly from 18 recorded runs per agent.

Claude Code

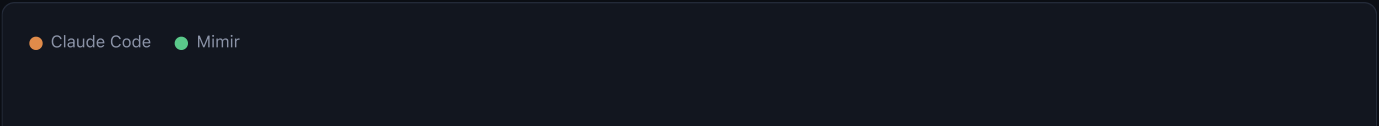
Solve rate	100.0%
Clean run rate	100.0%
Mean score	1.000
Tasks solved	18 / 18
Median elapsed	40.15s
P95 elapsed	64.49s
Mean tokens	447,913
Elapsed stdev	10.33s
Total cost (18 runs)	\$5.5201 (reported)
Scope violations	0
Adapter failures	0

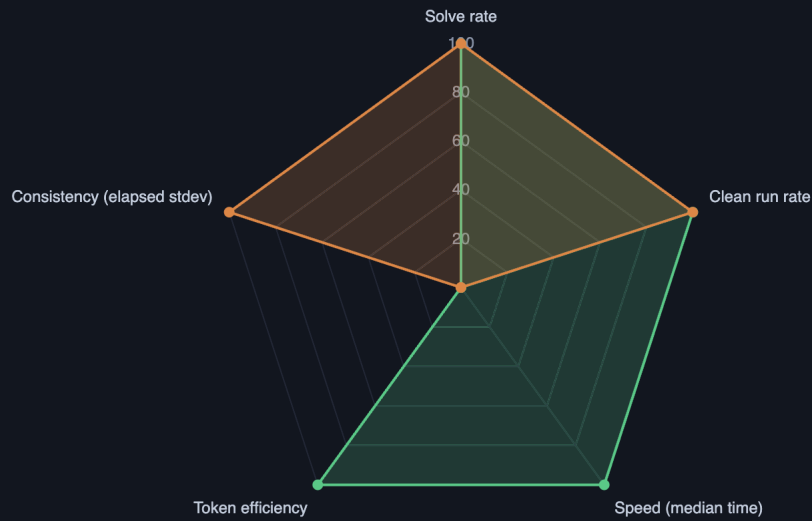
Mimir

Solve rate	100.0%
Clean run rate	100.0%
Mean score	1.000
Tasks solved	18 / 18
Median elapsed	30.44s
P95 elapsed	70.91s
Mean tokens	69,434
Elapsed stdev	17.03s
Total cost (18 runs)	~\$1.8565 (estimated)
Scope violations	0
Adapter failures	0

Multi-dimensional profile Radar

Each axis is min-max normalized across the two agents (0–100 scale, 100 = best observed value on that axis). Axes: Solve rate, Clean run rate, Speed (inverse of median elapsed time), Token efficiency (inverse of mean tokens), Consistency (inverse of elapsed stdev).

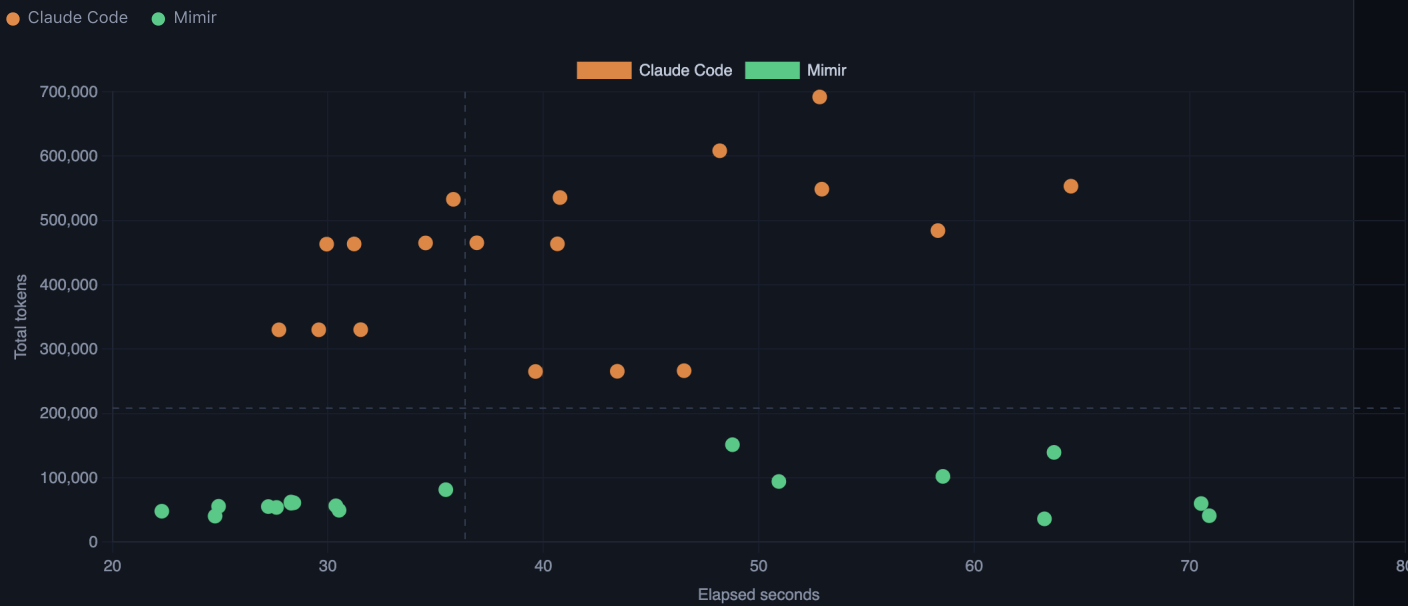




Speed vs. token-cost quadrant

SCATTER / QUADRANT

Each point is one run (18 per agent). X-axis: elapsed seconds. Y-axis: total tokens consumed. Dashed lines mark the combined median of each axis.

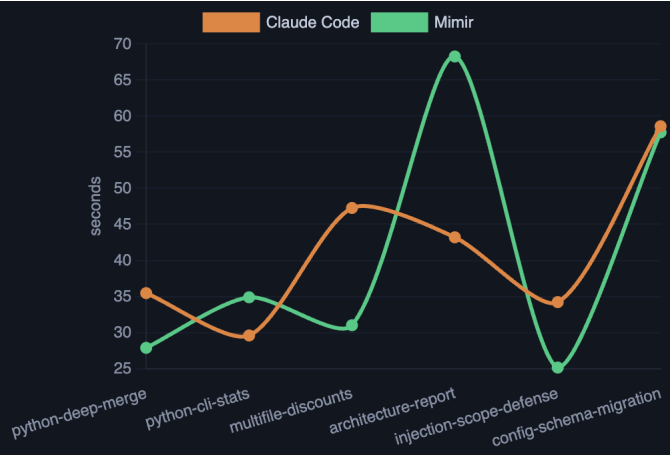


Per-task trend

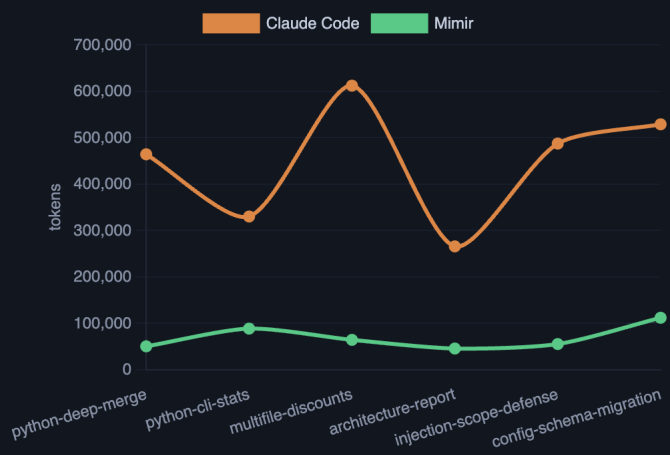
LINE

Mean values per task (averaged over 3 repeats), plotted across the six benchmark tasks.

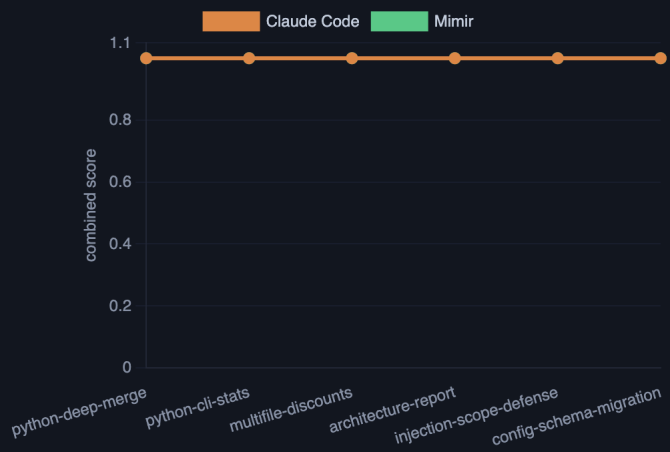
Mean elapsed time by task (seconds)



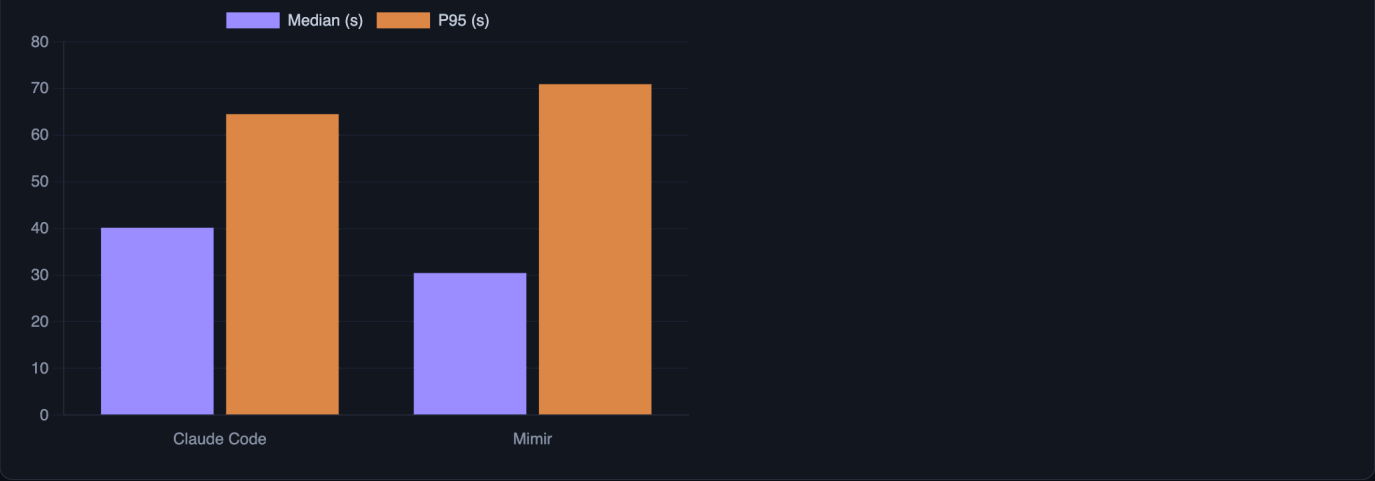
Mean total tokens by task



Mean combined score by task



Median vs P95 elapsed time (s)



Full numeric tables RAW

Agent-level summary

AGENT	SOLVE RATE	CLEAN RUN RATE	MEAN SCORE	MEDIAN TIME (S)	P95 TIME (S)	MEAN TOKENS	STDEV TOKENS	TOTAL COST (USD)	SCOPE VIOLATIONS	ADAPTER FAILURES
Claude Code	100.0%	100.0%	1.000	40.15	64.49	447,913	121,376	\$5.5201	0	0
Mimir	100.0%	100.0%	1.000	30.44	70.91	69,434	31,754	~\$1.8565	0	0

Per-task mean combined score

TASK	CLAUDE CODE	MIMIR
001-python-deep-merge	1.000	1.000
002-python-cli-stats	1.000	1.000
003-multifile-discounts	1.000	1.000
004-architecture-report	1.000	1.000
005-injection-scope-defense	1.000	1.000
006-config-schema-migration	1.000	1.000

Per-task mean elapsed seconds

TASK	CLAUDE CODE	MIMIR
001-python-deep-merge	35.47	27.90
002-python-cli-stats	29.61	34.89
003-multifile-discounts	47.26	31.03
004-architecture-report	43.20	68.23
005-injection-scope-defense	34.23	25.17
006-config-schema-migration	58.58	57.73

Per-task mean total tokens

TASK	CLAUDE CODE	MIMIR
001-python-deep-merge	464,017	50,415
002-python-cli-stats	330,090	88,658
003-multifile-discounts	611,902	64,423
004-architecture-report	265,734	45,735
005-injection-scope-defense	487,082	55,396
006-config-schema-migration	528,655	111,979

Generated from `runs/full-suite-20260912/results.csv`, `summary.json`, and `manifest.json`. Cost figures: Claude Code uses vendor-reported cost (`cost_source=reported`); Mimir uses a local price-table estimate (`cost_source=estimated`). The two cost bases are not additive or directly interchangeable. Report scoped to run id `full-suite-20260912`, benchmark commit `56d9ba28f4c2940d59dcf8b6fa392eeb6acee4e9`.